Notation

The notation used in this book is fairly standard and should require little explanation. We freely use vector and matrix notation, generally using upper-case bold type for matrices, lower-case bold type for vectors, and regular type for scalars. Iteration and component indices are denoted by subscripts, usually i through n. For example, a vector $\boldsymbol{x}$ and matrix $\boldsymbol{A}$ have entries x_i and a_{ij}, respectively. On the few occasions when both an iteration index and a component index are needed, the iteration is indicated by a parenthesized superscript, as in $x_i^{(k)}$ to indicate the ith component of the kth vector in a sequence. Otherwise, x_i denotes the ith component of a vector $\boldsymbol{x}$, whereas $\boldsymbol{x}_k$ denotes the kth vector in a sequence.

For simplicity, we will deal primarily with real vectors and matrices, although most of the theory and algorithms we discuss carry over with little or no change to the complex field. The set of real numbers is denoted by $\mathbb{R}$, n-dimensional real Euclidean space by $\mathbb{R}^n$, and the set of real $m \times n$ matrices by $\mathbb{R}^{m\times n}$. The analogous complex entities are denoted by $\mathbb{C}$, $\mathbb{C}^n$, and $\mathbb{C}^{m\times n}$, respectively.

The transpose of a vector or matrix is indicated by a superscript T, and the conjugate transpose by superscript H (for Hermitian transpose). Unless otherwise indicated, all vectors are regarded as column vectors; a row vector is indicated by explicitly transposing a column vector. For typesetting convenience, the components of a column vector are sometimes indicated by transposing the corresponding row vector, as in $\boldsymbol{x} = [\, x_1 \quad x_2 \,]^T$. The inner product (also known as dot product or scalar product) of two n-vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ is a special case of matrix multiplication and thus is denoted by $\boldsymbol{x}^T\boldsymbol{y}$ (or $\boldsymbol{x}^H\boldsymbol{y}$ in the complex case). Similarly, the outer product of two n-vectors $\boldsymbol{x}$ and $\boldsymbol{y}$, which is an $n \times n$ matrix, is denoted by $\boldsymbol{x}\boldsymbol{y}^T$. The identity matrix of order n is denoted by $\boldsymbol{I}_n$ (or just $\boldsymbol{I}$ if the dimension n is clear from context), and its ith column is denoted by $\boldsymbol{e}_i$. A zero matrix is denoted by $\boldsymbol{O}$, a zero vector by $\boldsymbol{0}$, and a zero scalar by 0. A diagonal matrix with diagonal entries $d_1, \ldots, d_n$ is denoted by $\operatorname{diag}(d_1, \ldots, d_n)$. Inequalities between vectors or matrices are taken to apply elementwise. The subspace of $\mathbb{R}^m$ spanned by the columns of an $m \times n$ matrix $\boldsymbol{A}$, i.e., $\{\boldsymbol{A}\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^n\}$, is denoted by $\operatorname{span}(\boldsymbol{A})$.

The ordinary derivative of a function $f(t)$ of one variable is denoted by df/dt or by $f'(t)$. Partial derivatives of a function of several variables, such as $u(x, y)$, are denoted by $\partial u/\partial x$, for example, or in some contexts by a subscript, as in u_x. Notation for gradient vectors and Jacobian and Hessian matrices will be introduced as needed. All logarithms are natural logarithms (base $e \approx 2.718$) unless another

base is explicitly indicated. We use the symbol $\approx$ to indicate approximate equality in the ordinary sense and reserve the symbol $\cong$ specifically for least squares approximations.

The computational cost, or *complexity*, of numerical algorithms is usually measured by the number of arithmetic operations required. Traditionally, numerical analysts have counted only multiplications (and possibly divisions and square roots), because multiplications were usually significantly more expensive than additions or subtractions and because in most algorithms multiplications tend to be paired with a similar number of additions (for example, in computing the inner product of two vectors). More recently, the difference in cost between additions and multiplications has largely disappeared (indeed, many modern microprocessors can perform a coupled multiplication and addition with a single `multiply-add` instruction). Computer vendors and users like to advertise the highest possible performance, so it is increasingly common for every arithmetic operation to be counted. Because certain operation counts are so well known using the traditional practice, however, only multiplications are usually counted in this book. To clarify the meaning, the phrase "and a similar number of additions" will be added, or else it will be explicitly stated when both are being counted.

In quantifying operation counts and the accuracy of approximations, we will often use "big-oh" notation to indicate the order of magnitude, or dominant term, of a function. For an operation count, we are interested in the behavior as the size of the problem, say n, becomes large. We say that

$$f(n) = \mathcal{O}(g(n))$$

(read "f is big-oh of g" or "f is of order g") if there is a positive constant C such that

$$|f(n)| \leq C|g(n)|$$

for all n sufficiently large. For example,

$$2n^3 + 3n^2 + n = \mathcal{O}(n^3)$$

because as n becomes large, the terms of order lower than n^3 become relatively insignificant. For an accuracy estimate, we are interested in the behavior as some quantity h, such as a step size or mesh spacing, becomes small. We say that

$$f(h) = \mathcal{O}(g(h))$$

if there is a positive constant C such that

$$|f(h)| \leq C|g(h)|$$

for all h sufficiently small. For example,

$$\frac{1}{1-h} = 1 + h + h^2 + h^3 + \cdots = 1 + h + \mathcal{O}(h^2)$$

because as h becomes small, the omitted terms beyond h^2 become relatively insignificant. Note that the two definitions are equivalent if $h = 1/n$.